

AUDITORÍA ARQUITECTÓNICA · ETAPA I

Capítulos 8–12 — Hardware, Servicios Compartidos, Procesos, Eventos e Integraciones

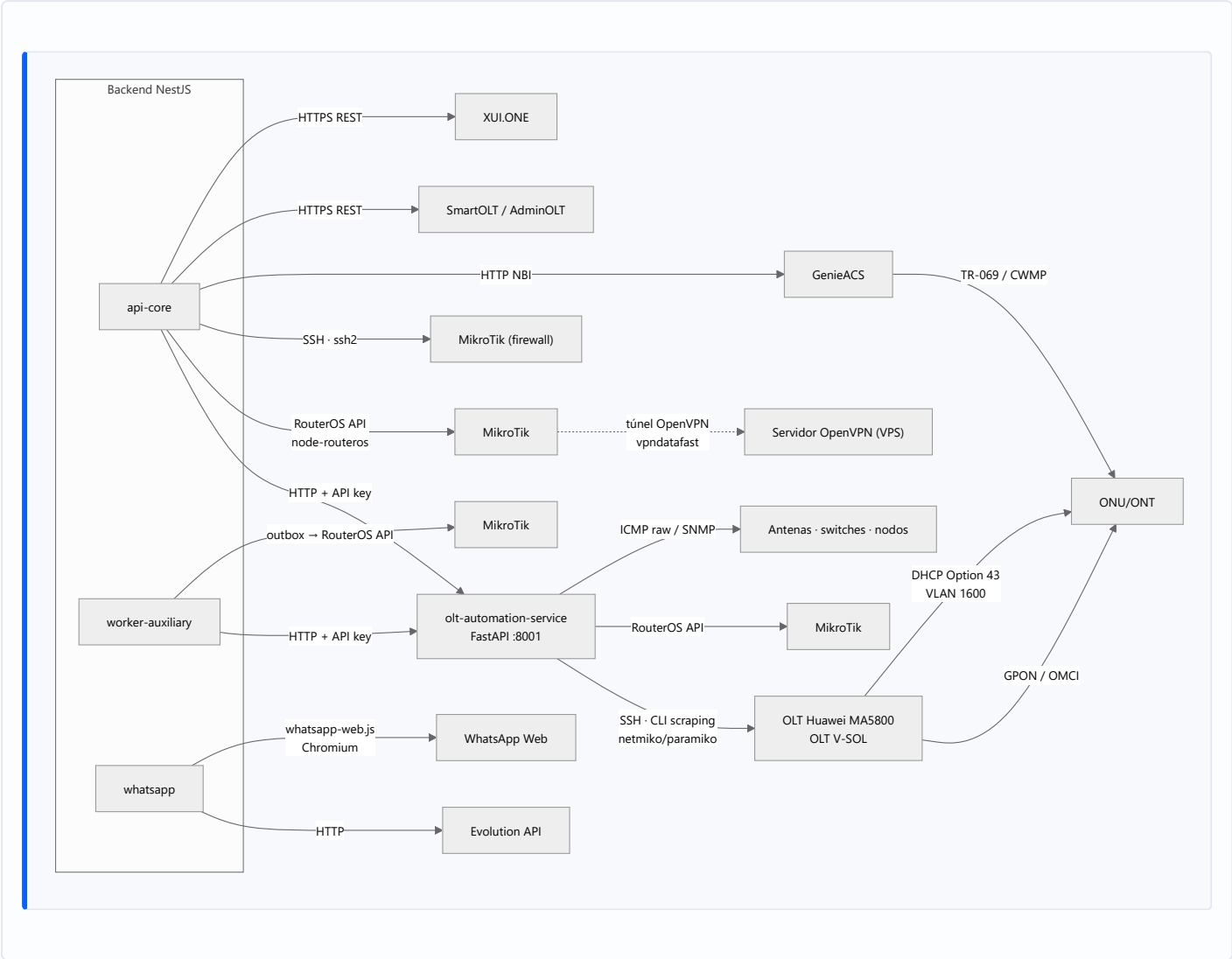
ERP Datafast ISP

Capítulos 8–12 — Hardware, Servicios Compartidos, Procesos, Eventos e Integraciones

Fecha de levantamiento: 2026-08-06 · **Rama:** main · **Commit base:** f8d52b00
Alcance: documenta el estado actual del sistema. No propone rediseños ni modifica código.
Documento: Hardware, Servicios Compartidos, Procesos, Eventos e Integraciones

CAPÍTULO 8 — Comunicación con Equipos

8.1 Mapa de transportes



8.2 OLT Huawei MA5800 / V-SOL

Aspecto	Detalle medido
Quién consulta	Únicamente <code>olt-automation-service</code> (Python). El backend NestJS nunca abre una sesión SSH a una OLT .
Protocolo	SSH — scraping de CLI (no TL1, no NETCONF, no SNMP para configuración)
Driver	<code>app/drivers/huawei.py</code> , <code>app/drivers/vsol.py</code> , contrato en <code>base.py</code>
Lógica	<code>app/services/provisioning.py</code> — 4.792 LOC, el archivo más grande del repositorio
Pool de sesiones	<code>app/services/connection_pool.py</code> (96 LOC)
Concurrencia	uvicorn con <code>--workers 1</code> deliberado — el MA5800 tiene un límite bajo de sesiones VTY concurrentes. Cada worker abriría sus propias sesiones SSH.

Aspecto	Detalle medido
<code>--reload</code> prohibido	Comentado explícitamente en <code>ecosystem.config.js</code> : WatchFiles reiniciaba uvicorn al tocar cualquier archivo, y un <code>git reset --hard</code> de deploy lo disparó en medio de una Fase 2 WAN → ONT huérfano (2026-07-21).

Qué información se obtiene

Operación	Comando/lectura	Dónde se almacena	Quién la reconsulta
Inventario de ONUs	<code>display ont info</code> (autofind + registradas)	<code>olt_onu_inventario</code>	<code>olt-inventario-refresh.service.ts</code> por evento <code>ftth.inventario.reobservevar</code> ; sync 6 h
Potencia óptica	<code>display ont optical-info</code>	<code>metricas_onu_optical</code>	Dashboard de señal, modal ONU
Topología de tarjetas	<code>display board</code>	<code>olt_boards</code>	Wizard, health
Perfiles	<code>display ont-lineprofile/srvprofile</code> , <code>traffic-table</code>	<code>olt_line_profiles</code> , <code>olt_service_profiles</code> , <code>olt_traffic_tables</code>	Compliance, baseline
VLANs	<code>display vlan</code>	<code>olt_vlans</code>	Baseline, sincronización
Versión de ONT	<code>display ont version</code>	<code>olt_onu_inventario</code>	Catálogo de modelos
Salud (POM, PON, boards)	varios <code>display</code>	<code>olt_health_snapshots</code>	<code>olt-health-poller.cron.ts</code>
IDs ocupados en PON	<code>/ftth/ont-ids</code>	<code>olt_onu_id_pool</code>	Reconciliación de pool

Operaciones mutantes y su verificación VIO

Operación	Escritura	Verificación independiente (VIO)	Timeout
Registro GPON	<code>ont add</code> (descripción <code>DATAFAST_CNT-xxxx</code>)	<code>poll-online</code> + <code>verify-onu</code>	—
Inyección WAN PPPoE	comando WAN	<code>check_ont_wan_pppoe</code>	90 s (era 30 s; causó el ONT huérfano del 21/07 16:55)
Carril de gestión TR-069	<code>ont ipconfig ... dhcp vlan 1600</code> + <code>ont tr069-server-config</code>	<code>check_ont_mgmt_ip</code> + convergencia ACS	—
Rollback GPON	<code>ont delete</code>	loop de verificación	150 s (era 30 s)
Undo service-port	<code>undo service-port</code>	<code>_undo_service_port_verificado</code>	—
Suspensión / rehabilitación	<code>ont deactivate</code> / <code>activate</code>	loop de verificación	30 s → causó latencia de 287 s en REACTIVAR; corregido

Hallazgo VIO fundacional (incidente 2026-07-17, CNT-2026-000004): una ONU Huawei EG8145V5 aceptó sin error el comando OMCI del carril de gestión TR-069 —la OLT lo mostraba configurado— pero el firmware nunca activó el IP-host (0 tramas Ethernet, confirmado con sniffer en cold-boot físico). El ERP reportó "carril aplicado" durante días con la gestión remota muerta, porque solo verificaba que el CLI no devolviera error. De ahí la regla `accepted ≠ materialized` .

Hallazgos de firmware documentados (R018)

- `dba-profile delete profile-name` (no `undo`).
- Eco pegado = problema de sintaxis, no de transporte.
- **Hay que drenar el autosave antes de enviar comandos**; un reintento CLI tras autosave produce `% Unknown command` y genera falsos negativos (corregido con `confirmarConvergencia` antes del rollback, commit `ae07e733`).
- `ont reset` **NO reinicia una EG8145V5** — probado con captura de tráfico (0 paquetes a la OLT). SmartOLT usa TR-069 al CPE. Un power-cycle físico sí gatilla boot-inform; `ont reset` no.

8.3 MikroTik RouterOS

Dos caminos independientes hacia el mismo hardware:

Camino	Implementación	Uso
NestJS directo	<code>node-routeros</code> vía <code>mikrotik/services/connection-pool.service.ts</code>	Operaciones interactivas del operador (<code>/mikrotik/*</code>)
NestJS por SSH	<code>ssh2</code> en <code>mikrotik/services/firewall.service.ts</code>	Configuración de firewall
Python	<code>mikrotik_pool.py</code> + <code>mikrotik_ops.py</code> vía <code>/api/v1/mikrotik/*</code>	Operaciones invocadas desde el servicio de automatización

Aspecto	Detalle
Alcanzabilidad	Exclusivamente a través del túnel OpenVPN <code>vpndatafast</code> . Sin VPN no hay gestión.
Servicios NestJS	<code>pppoe</code> , <code>queue</code> , <code>firewall</code> , <code>arp</code> , <code>interface</code> , <code>wireless</code> , <code>subnet-route</code> , <code>mikrotik-user</code> , <code>address-list-reconciliador</code> , <code>services/velocidad/</code>
Fix conocido	Doble patch <code>!empty</code> (Channel + Receiver) en <code>connection-pool.service.ts</code> — resolvió un timeout de 30 s → 988 ms
Circuit breaker	Por router: <code>GET /mikrotik/routers/:id/circuit-breaker</code> , <code>POST /circuit-breaker/reset</code>
Drift	<code>GET /mikrotik/drift</code> , <code>GET /mikrotik/address-lists/sobrantes</code> , tabla <code>drift_detectado</code>

Qué información se obtiene y con qué frecuencia

Dato	Endpoint	Frecuencia	Almacenamiento
Estado del router	<code>GET /routers/:id/estado</code>	Bajo demanda + monitoreo	<code>routers</code> , <code>alertas_sistema</code>
Interfaces	<code>GET /routers/:id/interfaces</code>	Bajo demanda	No persistido
Tráfico	<code>GET /routers/:id/trafico</code>	Bajo demanda (vista en vivo)	No persistido
Sesiones PPPoE activas	<code>GET /routers/:id/sesiones</code>	Bajo demanda	No persistido
Queues	<code>GET /routers/:id/queues</code>	Bajo demanda + <code>discrepancias</code>	Comparado con <code>contratos.plan</code>
DHCP leases	<code>GET /routers/:id/dhcp</code>	Bajo demanda	No persistido

Dato	Endpoint	Frecuencia	Almacenamiento
Address-list de morosos	<code>GET /routers/:id/morosos</code>	Bajo demanda + cron 04:40	<code>drift_detectado</code>
Consumo por contrato	colector del portal	Cada 15 min	<code>consumo_datos</code> , <code>consumo_snapshot</code>

Operaciones mutantes

Provisión de abonado, suspensión, reactivación, amarre IP-MAC, cambio de velocidad de queue, configuración de firewall, sincronización de subredes. **Todas las mutaciones disparadas por el negocio (corte/reactivación) pasan por el outbox `comandos_red_pendientes`** ; las disparadas por el operador desde `/red/routers` son síncronas.

8.4 ONU / ONT y TR-069 (GenieACS)

Aspecto	Detalle
Quién consulta	<code>olt-nativo</code> vía NBI HTTP de GenieACS (<code>GENIEACS_NBI_URL</code>)
Camino de datos	ONU → CWMP/TR-069 → GenieACS → NBI → backend
Carril de gestión	VLAN 1600, IP por DHCP, ACS URL entregada por DHCP Option 43 desde el MikroTik
Estado	Gestión TR-069 resuelta end-to-end el 2026-07-19: carril DHCP + <code>ont tr069-server-config</code> + preset GenieACS <code>PeriodicInform=300</code> + provision <code>erp-connreq-creds</code> . Validado con power-cycle físico.
Modelo bajo demanda	El carril NO se inyecta con la provisión (<code>INJECTAR_CARRIL_AUTOMATICO</code> off). Se activa/desactiva desde "Ver ONU". Máquina de estados <code>ftth_carril_estado</code> , barrido TTL 3 días.
Operaciones	info, refresh, reboot, factory-reset, WiFi (por banda), PPPoE, acceso web
Almacenamiento	<code>tr069_device</code> , <code>contrato_onu_config</code> , <code>cpe_provisioning_attempt</code> , <code>cpe_web_credential</code>
Watchers	<code>tr069-cpe-drift-watcher.cron.ts</code> : staleness (<code>12,42 * * * *</code>), drift (<code>5-59/20</code>), barrido TTL (04:20), des-endurecimiento de auth residual (03:40)

Objetivo abierto documentado

Lograr la ACS URL por OMCI. Hoy solo converge Option 43, lo que ata el diseño a Huawei + un DHCP por VLAN. Confirmado con ONU de fábrica que el ME137 **no** escribe la URL en la EG8145V5. El DHCP del MikroTik **no es retirable**. Lección registrada: probar bootstrap sobre un equipo que ya tiene la config buscada da falso positivo.

Incidente de deadlock de autenticación (resuelto)

La muerte de gestión TR-069 tras un factory-reset no era de IP ni de Option 43: el device conservaba el tag `AuthEnforced` mientras la ONU reseteada informaba sin credenciales → GenieACS rechazaba el Inform (HMAC no coincidía) → deadlock. Probado con `tcpdump` . Fix aplicado y validado (commit `99d9ad62`) : gracia de bootstrap — `factoryReset` y `reconcilePendingReinjection` quitan el tag; `enforceDeviceAuth` re-endurece al re-provisionar.

8.5 OpenVPN

Aspecto	Detalle
Rol	Único canal de alcanzabilidad hacia los MikroTik de campo
Quién administra	Módulo <code>openvpn</code> (backend), sobre el filesystem del VPS
Artefactos	Certificados por router, CCD con <code>ifconfig-push</code> e <code>iroute</code> , script de cliente MikroTik
Ciclo de vida de IP	Permanente. <code>validarTunnel</code> escribe el CCD en el primer handshake (bloqueo inmediato). Solo se libera al eliminar el router (<code>removeRouter</code> → revoca certs → borra CCD → mata el túnel) o al cancelar el wizard sin completar el paso 3 (<code>fireRevoke</code>).
Red de seguridad	Cron <code>limpiarWizardsAbandonados</code> cada 30 min, corte a 2 h
Semántica de <code>iroute</code>	Declara propiedad de una subred (sale de <code>subnets_locales</code>), no alcanzabilidad. A nivel OSPF todo se alcanza; a nivel ERP el modelo es de propiedad. Dos routers reclamando la misma red no falla ruidosamente: da la respuesta equivocada con naturalidad.
Callbacks del servidor	El propio OpenVPN llama al backend: <code>verify-auth</code> , <code>verificar-sesion-cn</code> , <code>disconnect-notify</code>
Regla de script	Un script VPN por wizard, nunca regenerable ; la edición solo permite visualizar el script original
Incidente cerrado	Dar de baja un router dejaba la interfaz <code>vpndatafast</code> viva en el MikroTik reintentando cada 15 s indefinidamente ("router zombi"). Resuelto. Las rutas <code>10.0.0.0/8</code> del VPS no son huérfanas: son del <code>mikrotik.conf</code> .

8.6 Monitoreo de dispositivos (ICMP / SNMP)

Aspecto	Detalle
Quién consulta	<code>monitoreo-worker.service.ts</code> (NestJS, cron cada minuto) → <code>olt-automation-service</code> <code>/api/v1/monitoring/*</code>
Protocolos	ICMP raw (<code>icmplib</code> , contenedor con <code>cap_add: NET_RAW</code>), SNMP (<code>snmp_service.py</code> , <code>snmp_mapping.py</code>)
Objetivos	Antenas, switches, nodos, routers — tabla <code>dispositivos_monitoreo</code>
Almacenamiento	<code>metricas_monitoreo</code> (alta escritura), <code>alertas_sistema</code> , <code>nodos_mediciones</code>
Salida	Vista <code>v_estado_dispositivos</code> , WebSocket <code>monitoreo.gateway.ts</code> , eventos <code>ROUTER_CAIDO</code> / <code>ROUTER_CONECTADO</code>
Umbrales	Tabla <code>umbrales_alerta</code> , configurables por endpoint
<code>net-snmp</code> en Node	La dependencia está declarada en <code>backend/package.json</code> , pero el polling SNMP real vive en Python.

8.7 SmartOLT / AdminOLT

Aspecto	Detalle
Protocolo	HTTPS REST (<code>SMARTOLT_URL</code> , <code>SMARTOLT_TOKEN</code>)

Aspecto	Detalle
Rol	Proveedor alternativo de operaciones OLT, enrutado por <code>olt-provider-registry.service.ts</code> + <code>olt-operation-router.service.ts</code>
Configuración	<code>olt_proveedor_config</code> , con circuit breaker (<code>POST /proveedores/:configId/reset-circuit</code>)
Estado	Camino legado. El nodo real (NODO MALVINAS, 205 ONUs) está en migración a <code>olt-nativo</code> ; paso 1 (reconciliar pools) completado.

8.8 XUI.ONE (IPTV)

Aspecto	Detalle
Protocolo	HTTP REST
Módulo	<code>xui</code> — degradable por construcción
Datos	Bouquets, líneas, estado de canales — <code>xui_servidores</code> , <code>xui_lines</code>
Frecuencia	<code>xui-monitor.service.ts</code> — cron cada 30 segundos (el más frecuente del sistema)

8.9 Equipos y protocolos NO integrados

Para evitar suposiciones: el prompt de auditoría mencionaba equipos que **no existen en este sistema**.

Elemento	Estado real
Astra	No integrado. Sin referencias en el código.
UPS	No integrado. Sin lectura de estado de UPS.
Telnet	No usado. Todo el acceso a OLT es SSH.
TL1	No usado.
SOAP	No usado. TR-069/CWMP es SOAP, pero lo habla GenieACS, no el ERP.
Switches gestionados	Solo como objetivo genérico de <code>dispositivos_monitoreo</code> (ICMP/SNMP). Sin driver de configuración.
NETCONF / RESTCONF	No usado.

CAPÍTULO 9 — Servicios Compartidos

9.1 Servicios globales (registrados en AppModule , inyectables sin importar)

Servicio	Archivo	Función	Consumidores
ModuleHealthService	common/services/module-health.service.ts	Registro de estado ok / degraded por módulo con razón. Base del patrón degradable.	Todos los módulos degradables + GET /health/modules
WatcherHeartbeatService	common/services/watcher-heartbeat.service.ts	Latido de los procesos de fondo	Crons y watchers; GET /admin/sistema/watchers
RedisLockService	common/redis/redis-lock.service.ts	Locks distribuidos y circuit breakers sobre Redis	Solo workers/cobranza.worker.ts (medido)
CircuitBreakerRegistry	common/services/circuit-breaker.registry.ts	Registro central de breakers	mikrotik , olt-nativo (proveedores), monitoreo
QueuePauseService	common/services/queue-pause.service.ts	Pausa/reanudación de las 4 colas principales	mantenimiento , sistema
CacheModule (global)	@nestjs/cache-manager + Redis db=0	Cache de aplicación, TTL 300 s	15 archivos (ver Cap. 14)
EventEmitterModule (global)	@nestjs/event-emitter	Bus de eventos in-process, maxListeners: 30	25 listeners
ConfigModule (global)	@nestjs/config con validación Joi	Configuración	Todos

9.2 Piezas transversales aplicadas globalmente

Pieza	Tipo	Efecto
LicenciaGuard	APP_GUARD #1	Bloquea todo el ERP sin licencia válida
JwtAuthGuard	APP_GUARD #2	Autenticación en cada request salvo @Public()
RolesGuard	APP_GUARD #3	Autorización por rol / @RequirePermission
ThrottlerGuard	APP_GUARD #4	Rate limiting 10/s · 100/min · 1000/h
LoggingInterceptor	APP_INTERCEPTOR	Log de request/response (winston)
TimeoutInterceptor(30 s)	APP_INTERCEPTOR	Corta toda request a 30 s
AuditInterceptor	APP_INTERCEPTOR	Escribe auditoria_logs + entity_versions ; respeta skipAudit
TransformInterceptor	APP_INTERCEPTOR	Envoltura uniforme de respuesta
AllExceptionsFilter	APP_FILTER	Normalización de errores

Interacción crítica medida: TimeoutInterceptor(30000) es global. Operaciones contra hardware con timeouts internos de 90 s (inyección WAN) y 150 s (rollback GPON) **exceden el timeout HTTP global**. El diseño lo resuelve haciendo esas operaciones asíncronas (outbox + watcher), no ampliando el timeout.

9.3 Utilidades compartidas

Utilidad	Archivo	Uso
<code>encryption.util.ts</code>	<code>common/utils/</code>	Cifrado de credenciales (<code>ENCRYPTION_KEY</code>) — routers, OLTs, proveedores, tokens Google
<code>ip.util.ts</code>	<code>common/utils/</code>	Cálculo de subredes, siguiente IP, validación CIDR
<code>pagination.util.ts</code>	<code>common/utils/</code>	Paginación uniforme
<code>pg-result.util.ts</code>	<code>common/utils/</code>	Normalización de resultados de SQL crudo
<code>telefono.util.ts</code>	<code>common/utils/</code>	Normalización de teléfonos (envío de mensajería)
<code>resultado-operacion.ts</code>	<code>common/domain/</code>	Vocabulario de dominio — 6 clases de resultado, <code>traducirAHttp</code> en el borde
<code>service-types.ts</code>	<code>common/constants</code> <code>/</code>	Tipos de servicio
<code>base.entity.ts</code>	<code>common/entities/</code>	Entidad base
<code>@CurrentUser</code> , <code>@Public</code> , <code>@Roles</code>	<code>common/decorator</code> <code>s/</code>	Decoradores de contexto y acceso

9.4 Servicios de dominio reutilizados entre módulos

Servicio	Módulo propietario	Reutilizado por
<code>MikrotikService</code> + pool de conexiones	<code>mikrotik</code>	contratos, monitoreo, openvpn, portal, promesas-pago, reconciliador, sites, smartolt, workers
<code>OutboxRedService</code>	<code>outbox-red</code>	contratos, promesas-pago, workers
<code>PoliticaFacturacionService</code>	<code>facturacion</code>	Fórmula única del ciclo de cobro (emisión/vencimiento/corte). Antes eran 3 fórmulas y el corte caía antes del vencimiento (incidente 05/08).
<code>GatewayMensajeriaService</code>	<code>notificaciones</code>	mensajería, sistema, finanzas-opex, workers
<code>ConfigService</code> de empresa	<code>config</code>	8 módulos
<code>AuthService</code> / estrategias	<code>auth</code>	11 módulos
<code>FtthOperacionLockService</code>	<code>olt-nativo</code>	Toda operación FTTH mutante
<code>OltProviderRegistry</code> + <code>OltOperationRouter</code>	<code>olt-nativo</code>	Enrutamiento nativo/SmartOLT/AdminOLT
CTE <code>PUNTOS_SERVICIO</code>	<code>planta-externa</code>	mapa de clientes, planta externa

9.5 Servicios compartidos que el prompt asumía y NO existen como tales

Asumido	Realidad medida
Servicio de archivos	No hay abstracción. Cada módulo usa <code>multer</code> + filesystem directamente (<code>clientes/:id/foto</code> , <code>config/empresa/logo</code> , <code>pagos/:id/comprobante</code> , media CRM).
Servicio de logs unificado	Hay winston configurado globalmente, pero cada módulo loguea a su manera ; no hay contrato de log estructurado.
Servicio de configuración de negocio	La configuración vive repartida: <code>empresas</code> , <code>configuracion_facturacion</code> , <code>portal_config</code> , <code>olt_onu_preset</code> , <code>openvpn_config</code> , <code>olt_proveedor_config</code> . No hay un servicio de settings único.
Servicio de permisos como librería	<code>RolesGuard</code> + <code>@RequirePermission</code> existen, pero la aplicación del decorador es parcial (§5.1).

CAPÍTULO 10 — Procesos Programados

10.1 Regla de ejecución

Todos los crons están declarados con `@Cron` de `@nestjsjs/schedule` dentro del mismo binario. `ScheduleModule.forRoot()` se registra en los tres procesos Node, pero cada servicio comprueba `RUN_CRONS` antes de añadir su tarea. En producción solo `datafast-worker-auxiliary` tiene `RUN_CRONS=true`.

No hay cron del sistema operativo (`crontab`) ejecutando lógica de negocio. El endpoint `GET/PATCH /admin/sistema/crontab` administra el crontab del VPS para tareas de plataforma (backups, certbot), no para el ERP.

10.2 Inventario completo (29 tareas)

#	Expresión	Nombre / método	Módulo	Objetivo	Dependencias
1	<code>* / 30 * * * *</code> <code>* (30 s)</code>	<code>xui-monitor tick()</code>	<code>xui</code>	Estado de canales IPTV	XUI.ONE
2	<code>* * * * * (1 min)</code>	<code>procesarVencidas()</code>	<code>promesas-pago</code>	Vencer promesas → dispara corte	outbox-red, mikrotik
3	<code>EVERY_MINUTE</code>	<code>runCycle()</code>	<code>monitoreo</code>	Ping/SNMP a todos los dispositivos	olt-automation-service, BD
4	<code>* / 2 * * * *</code>	<code>watchPendingReinjection()</code>	<code>olt-nativo</code>	Reinyección TR-069 pendiente	GenieACS, OLT
5	<code>2-59 / 3 * * * *</code> <code>*</code>	<code>procesarAnulaciones()</code>	<code>olt-nativo</code>	Anulación asíncrona de wizards	OLT, saga
6	<code>4-59 / 5 * * * *</code> <code>*</code>	<code>liberarBloqueados()</code>	<code>olt-nativo</code>	Liberar locks FTTH expirados	<code>ftth_operacion_lock</code>
7	<code>0 * / 5 * * * *</code> <code>*</code>	<code>barridoProgramado()</code>	<code>outbox-red</code>	Drenar <code>comandos_red_pendientes</code>	mikrotik, olt-nativo
8	<code>30 * / 5 * * * *</code> <code>*</code>	<code>barrerClaimsExpirados()</code>	<code>outbox-red</code>	Recuperar claims huérfanos	BD
9	<code>0 * / 5 * * * *</code> <code>*</code>	<code>reintentarPendientes()</code>	<code>promesas-pago</code>	Reintento de promesas	outbox-red
10	<code>5-59 / 10 * * * *</code> <code>* *</code>	<code>reintentarRollbacks()</code>	<code>olt-nativo</code>	Watcher <code>fallido_rollback</code> (invariante de atomicidad)	OLT
11	<code>* / 10 * * * *</code>	<code>verificarWan()</code>	<code>olt-nativo</code>	VIO de WAN inyectada	OLT
12	<code>* / 10 * * * *</code>	<code>notif-orphan-cleanup</code>	<code>mensajería</code>	Limpiar notificaciones EN_PROCESO huérfanas	BD
13	<code>0 * / 10 * * * *</code> <code>*</code>	<code>reconciliarPagosNoAplicados()</code>	<code>pagos</code>	Pagos sin aplicar a factura	facturacion
14	<code>3-59 / 15 * * * *</code> <code>* *</code>	<code>recolectar()</code>	<code>portal</code>	Consumo de datos por contrato	mikrotik

#	Expresión	Nombre / método	Módulo	Objetivo	Dependencias
15	<code>* / 15 * * * *</code>	<code>notif-reconciler</code>	<code>mensajería</code>	Reconciliar notificaciones pendientes	gateway
16	<code>0 * / 15 * * * *</code>	<code>reconciliar()</code>	<code>reconciliador</code>	Estado BD ↔ MikroTik. Sin cap ni lock	mikrotik
17	<code>5-59 / 20 * * * *</code>	<code>verificarDrift()</code>	<code>olt-nativo</code>	Drift de CPE TR-069	GenieACS
18	<code>12,42 * * * *</code>	<code>verificarStaleness()</code>	<code>olt-nativo</code>	Sesiones TR-069 rancias	GenieACS
19	<code>* / 30 * * * *</code>	<code>limpiarIdsHuerfanos()</code>	<code>olt-nativo</code>	Pool de ONU-ID	OLT
20	<code>0 * / 30 * * * *</code>	<code>reconciliarFtthOnu()</code>	<code>reconciliador</code>	Estado BD ↔ OLT	smartolt, olt
21	<code>7-59 / 30 * * * *</code>	<code>adoptarHuerfanas()</code>	<code>olt-nativo</code>	Watcher CREATE del invariante: reconstruye <code>ftth_onu_registro</code> desde inventario+pool+lectura viva	OLT
22	(30 min, doc.)	<code>limpiarWizardsAbandonados</code>	<code>openvpn</code>	Revoca certs de wizards abandonados (corte 2 h)	filesystem VPS
23	<code>0 * / 6 * * * *</code>	<code>recargaPeriodica()</code>	<code>licencia</code>	Recarga de licencia	Servidor de licencias
24	<code>50 * / 6 * * * *</code>	<code>syncPeriodico()</code>	<code>olt-nativo</code>	Sincronización de inventario OLT	OLT (SSH)
25	<code>EVERY_DAY_AT _3AM</code>	<code>validacionDiaria()</code>	<code>licencia</code>	Validación diaria	Servidor de licencias
26	<code>EVERY_DAY_AT _3AM</code>	<code>purgar()</code>	<code>auditoría</code>	Retención de auditoría	BD
27	<code>30 3 * * * *</code> (Lima)	<code>reconciliarDiario()</code> (ZTP)	<code>olt-nativo</code>	Reconcile de config de ONU. Reescribe SSID/clave/web de las ONUs en drift.	GenieACS, OLT
28	<code>40 3 * * * *</code> (Lima)	<code>desendurecerAuthResidual()</code>	<code>olt-nativo</code>	Quita <code>AuthEnforced</code> residual	GenieACS
29	<code>20 4 * * * *</code> (Lima)	<code>barrerTtl()</code>	<code>olt-nativo</code>	TTL del carril TR-069 (3 días)	GenieACS, OLT
30	<code>40 4 * * * *</code>	<code>reconciliarDiario()</code>	<code>mikrotik</code>	Address-lists de morosos	MikroTik

Ventana de madrugada — concentración medida

03:00	<code>licencia.validacionDiaria</code>	+	<code>auditoria.purgar</code>
03:30	<code>ZtpReconcileCron.reconciliarDiario</code>	←	reescribe config de ONUs en drift
03:40	<code>tr069.desendurecerAuthResidual</code>		
04:20	<code>tr069.barrerTtl</code>		
04:40	<code>mikrotik.address-list-reconciliador</code>		

Pendiente registrado: revisar el reconciliador nocturno — qué corre exactamente a las 03:30/03:40/04:20 y por qué urge, dado que `reconcile()` itera sin cap ni lock y el carril automático multiplicó su carga potencial.

10.3 Colas Bull (6 colas, Redis db=2)

Cola	Constante	Jobs	Processor	Concurrencia
cobranza	QUEUES.COBRANZA	detectar-morosos , suspender-contrato , reactivar-contrato , evaluar-prorroga , vencer-prorroga , procesar-pago	workers/cobranza.worker.ts	Por defecto
facturacion	QUEUES.FACTURACION	marcar-vencidas , generar-mensual , generar-facturas-empresa , generar-factura-contrato	facturacion/facturacion.worker.ts	Por defecto
notificaciones	QUEUES.NOTIFICACIONES	notif-envio (+ tipos de aviso)	mensajeria/mensajeria.worker.ts	5
campanas	QUEUES.CAMPANAS	campana-masiva	mensajeria/campanas.worker.ts	1 (goteo)
mikrotik-jobs	QUEUES.MIKROTIK	mk-suspender , mk-reactivar , mk-sync-velocidades	(declarada; el trabajo real va por outbox)	—
google-sync	QUEUES.GOOGLE_SYNC	google-sync-contact , -contacts-bulk , google-calendar-event , google-drive-backup , google-geocode-address	google-integration/processors/google-sync.processor.ts	Por defecto
mikrotik-velocidad	VELOCIDAD_QUEUE	sincronizar-router , cambiar-velocidad	mikrotik/velocidad.worker.ts	Por defecto

Política de reintentos

Global (`app.module.ts`): `attempts: 3` , backoff exponencial base 5 s, `removeOnComplete: 100` , `removeOnFail: 500` .

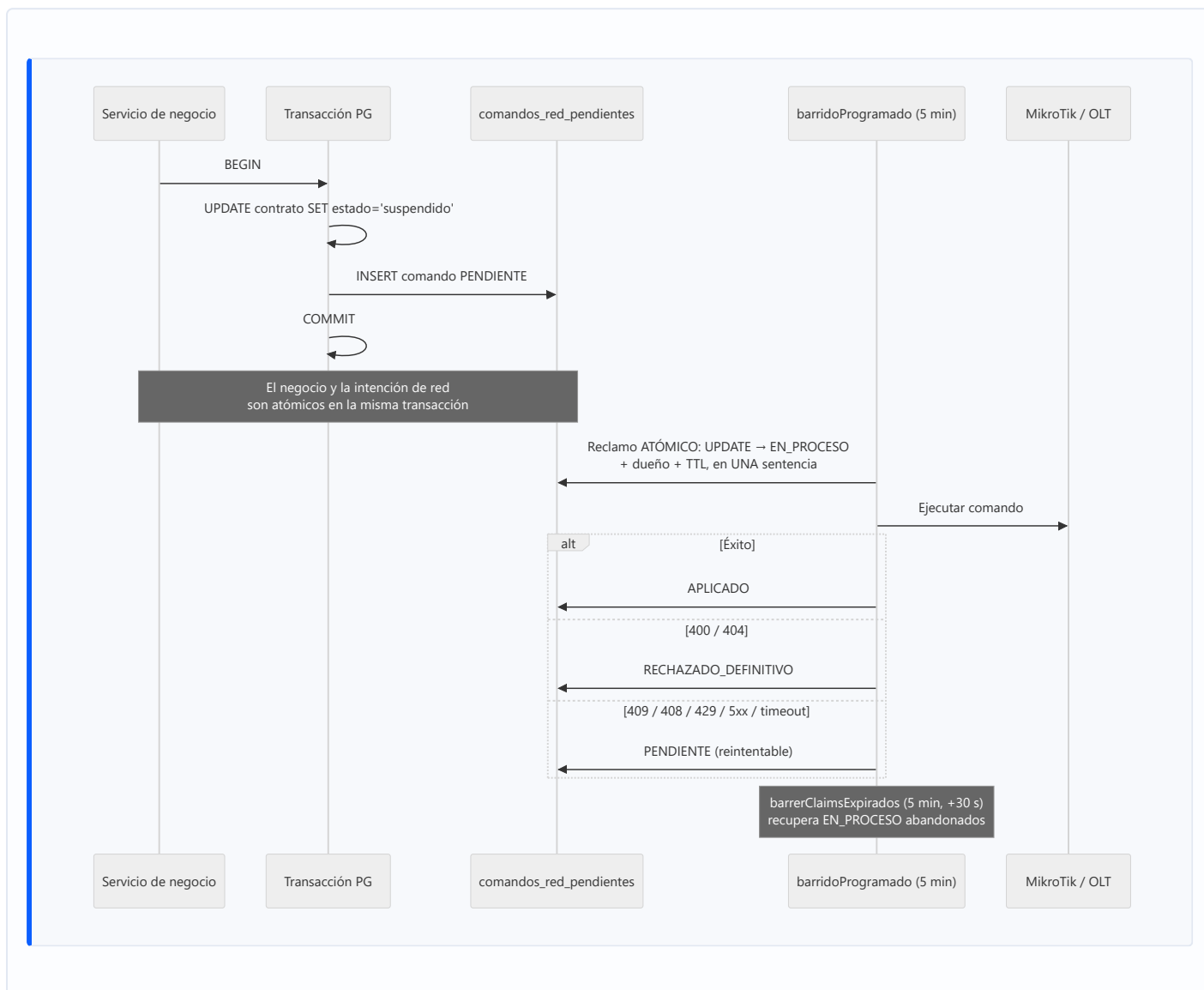
Por tipo (`workers.constants.ts` → `JOB_OPTIONS`):

Perfil	Prioridad	Intentos	Backoff
ALERTA	1	3	exponencial 15 s
SUSPENSION	2	2	fijo 60 s
AVISO_PAGO	3	2	fijo 60 s
CONFIRMACION_PAGO	4	2	fijo 60 s
GASTO_RECURRENTE	5	2	fijo 60 s
NOTIFICACION	10	2	fijo 60 s
CRITICO (aprovisionamiento)	—	3	exponencial 30 s
MIKROTIK	—	3	exponencial 10 s
MASIVO (facturación)	—	1	—

Goteo de campañas: `calcularDelayGoteo(i) = i*12.000 ms + random(0..4.000)` — ~5 mensajes por minuto máximo, con jitter.

10.4 El outbox de red — el tercer mecanismo asíncrono

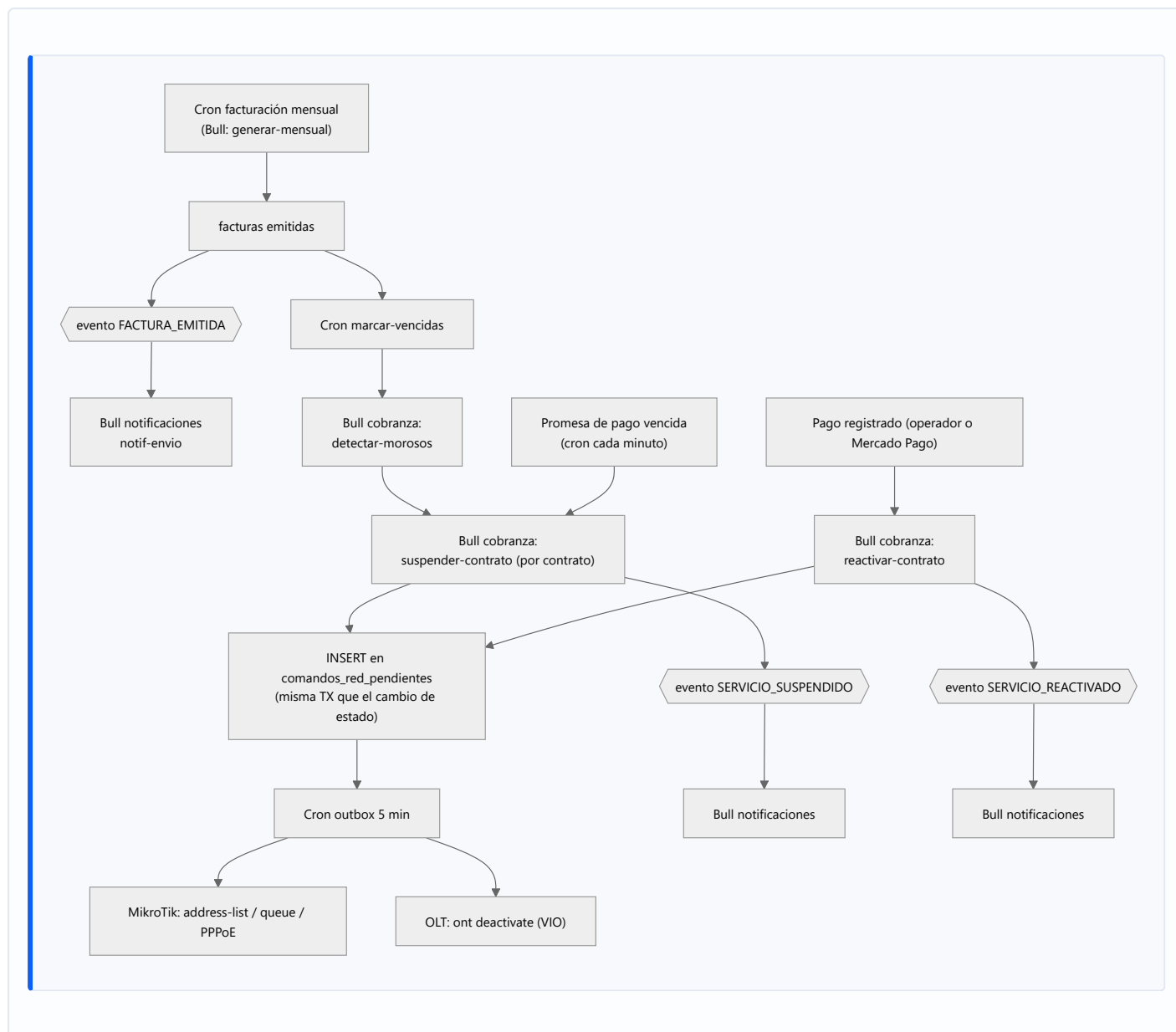
No es cola Bull ni cron: es una **tabla PostgreSQL** (`comandos_red_pendientes`) drenada por dos crons.



Por qué el reclamo es atómico: el diseño anterior comentaba que `SELECT FOR UPDATE SKIP LOCKED` garantizaba que "dos instancias PM2 nunca toman el mismo registro". Era **falso** — la transacción se cerraba antes de ejecutar contra el hardware, así que la exclusión protegía la selección pero no la ejecución. Nadie lo verificó; lo verifiqué producción. De ahí la regla "VIO hacia adentro": un comentario que garantiza concurrencia lleva un test que lo ejercite, o se borra.

Clasificador de reintentabilidad: solo `400` y `404` son definitivos. Un criterio amplio tipo `status < 500` es incorrecto — `409 / 408 / 429` significan "vuelve luego". Un `409` de lock leído como veredicto definitivo descartó trabajo; un no-op idempotente leído como fallo produjo **1.788 reintentos contra el MA5800 en 4 días**.

10.5 Cadena completa de cobranza (síncrono vs asíncrono)



Medición documentada: el "bucle" de la ONU al reactivar no era un bucle — eran dos latencias encadenadas: outbox sin drenado inmediato + timeout de 30 s en **rehabilitate**. REACTIVAR pasó de **287 s a 8 s**. El segundo defecto solo se hizo visible tras corregir el primero.

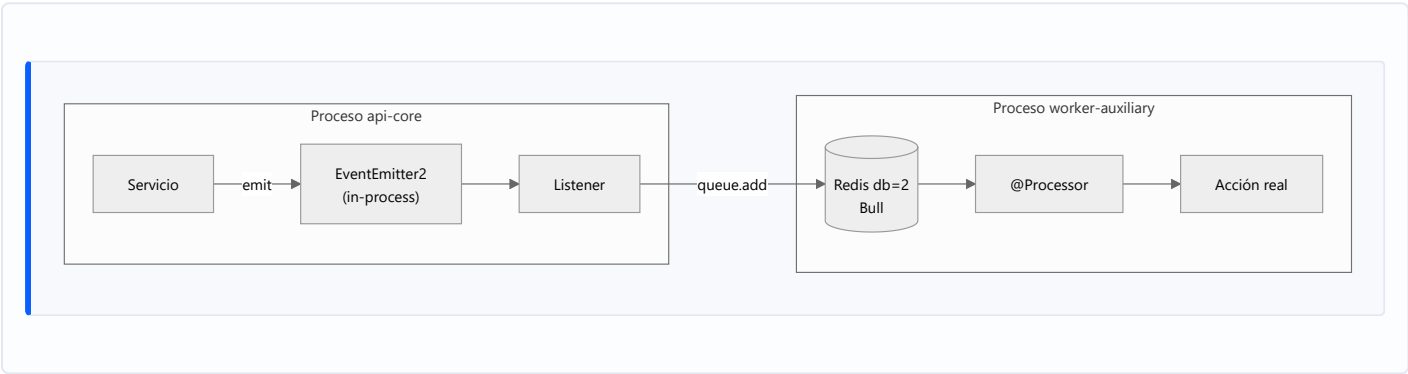
CAPÍTULO 11 — Eventos

11.1 Mecanismo

`EventEmitter2` vía `@nestjs/event-emitter`, configurado con `wildcard: false`, `delimiter: '.'`, `maxListeners: 30`, `ignoreErrors: false`.

Característica arquitectónica determinante: el bus es IN-PROCESS. No hay RabbitMQ, Kafka, NATS ni Redis Pub/Sub para eventos de dominio. Un evento emitido en `datafast-api-core` **no llega** a `datafast-worker-auxiliary`.

Los listeners no ejecutan trabajo directamente: **encolan en Bull**, que sí cruza procesos vía Redis. Ese es el puente entre el bus in-process y el worker.



11.2 Catálogo de eventos

Emitidos por `clientes`, `contratos`, `pagos`, `facturacion`, `mikrotik`, `monitoreo`

Evento	Emisor	Listeners
<code>cliente.created</code>	<code>clientes</code>	<code>google-events.listener</code> → sincroniza contacto Google
<code>instalacion.completed</code>	<code>contratos / aprovisionamiento</code>	<code>google-events.listener</code> → evento de calendario
<code>pago.registered</code>	<code>pagos</code>	<code>google-events.listener</code>
<code>visita.scheduled</code>	<code>tickets / contratos</code>	<code>google-events.listener</code>
<code>contrato.suspended</code>	<code>workers / contratos</code>	<code>google-events.listener</code>
<code>ftth.inventario.reobservar</code>	<code>olt-nativo</code>	<code>olt-inventario-refresh.service</code>
<code>GATEWAY_EVENTS.PROVIDER_ACTIVATED</code>	<code>notificaciones</code>	<code>mensajeria/gateway-monitor.service</code>
<code>OLT_SYNC_PROGRESS</code> / <code>OLT_SYNC_COMPLETED</code> / <code>OLT_SYNC_ERROR</code>	<code>olt-sync.service</code>	<code>olt.gateway</code> → retransmite a WebSocket

NOTIFICATION_EVENTS

 — 15 eventos escuchados por `notification-event.listener.ts`

Evento	Emisor típico	Acción
FACTURA_EMITIDA	facturacion	Encola <code>notif-envio</code> prioridad 2
PAGO_RECIBIDO	pagos	Prioridad 2
SERVICIO_SUSPENDIDO	workers	Prioridad 2
SERVICIO_REACTIVADO	workers	Prioridad 10
BIENVENIDA	clientes	Prioridad 10
PAGO_VENCE_HOY	workers	Prioridad 3
PAGO_VENCIDO	workers	Prioridad 3
PRORROGA_CONCEDIDA	promesas-pago	Prioridad 10
ALERTA_EGRESO	finanzas-opex	Prioridad 1
EMISOR_CAIDO / EMISOR_CONECTADO	mensajeria	Prioridad 1
ROUTER_CAIDO / ROUTER_CONECTADO	monitoreo	Prioridad 1
FTTH_ACTIVADO	olt-nativo	Prioridad 10
IPTV_LINE_CREADA	xui	Prioridad 10
OUTBOX_RED_AGOTADO	outbox-red	Alerta al operador: un comando de red agotó sus reintentos

11.3 WebSockets (3 gateways)

Gateway	Namespace/uso	Emite	Consumidor frontend
<code>olt.gateway.ts</code>	Progreso de sincronización OLT	Retransmisión de <code>OLT_SYNC_*</code>	<code>useOltSocket.ts</code> — <code>red/olt</code>
<code>monitoreo.gateway.ts</code>	Estado en vivo de dispositivos	Mediciones y alertas	<code>useMonitoreo.ts</code> — <code>monitoreo</code>
<code>crm-nativo.gateway.ts</code>	<code>/wa-socket</code>	Mensajes WhatsApp entrantes, QR de vinculación	<code>mensajeria/whatsapp</code>

Autenticación WS: `auth/guards/ws-jwt.guard.ts` .

11.4 Webhooks entrantes

Endpoint	Origen	Verificación
<code>POST /pagos/webhooks/mercadopago</code>	Mercado Pago	Firma del proveedor
<code>POST /webhooks/whatsapp</code>	Evolution API / Meta	Según configuración del gateway
<code>POST /admin/licencia/webhook/revocar</code>	Servidor de licencias	Clave de licencia

Endpoint	Origen	Verificación
POST /openvpn/mikrotik-clients/verify-auth · verificar-sesion-cn · disconnect-notify	Servidor OpenVPN local	Token / CN

11.5 Mensajería inter-servicio — declaración explícita

NO existe broker de mensajes en este sistema:

Tecnología	Estado
RabbitMQ	No usado, no declarado
Kafka	No usado, no declarado
NATS / MQTT	No usado
Redis Pub/Sub para eventos de dominio	No usado (Redis se usa para Bull, cache y locks)
gRPC	No usado
Event Sourcing	No implementado. <code>entity_versions</code> y <code>saga_log</code> son bitácoras, no un event store.

Los tres mecanismos asíncronos reales son: (1) colas Bull sobre Redis, (2) `EventEmitter2` in-process, (3) el outbox en PostgreSQL. La comunicación entre el backend y el servicio Python es **HTTP síncrono**.

CAPÍTULO 12 — Integraciones Externas

12.1 Inventario

#	Integración	Método	Módulo	Credencial	Frecuencia	Degradable
1	RENIEC	HTTPS REST	clientes (reniec.servi ce.ts)	RENIEC_API_URL , RENIEC_API_TOKEN	Bajo demanda (alta de cliente). Cacheada	Sí
2	Mercado Pago	REST + webhook	pagos (mercadopago. service.ts)	Credenciales del canal en BD (cifradas)	Bajo demanda + webhook	Sí
3	Google Calendar	OAuth2 + REST	google- integration	OAuth por empresa, token cifrado (GOOGLE_TOKEN_ENCRYPT ION_KEY)	Por evento (cola)	Sí
4	Google Contacts	OAuth2 + REST	google- integration	ídem	Por evento + bulk	Sí
5	Google Drive	OAuth2 + REST	google- integration	ídem	Backup encolado	Sí
6	Google Maps / Geocoding	REST	google- integration	GOOGLE_MAPS_API_KEY	Encolado (google- geocode-address)	Sí
7	OpenStreetMa p	Tiles vía MapLibre	Frontend	Ninguna	Render del mapa	—
8	Evolution API (WhatsApp)	HTTP + webhook	notificacione s , webhooks	EVOLUTION_API_URL , EVOLUTION_API_KEY	Por notificación	Sí
9	WhatsApp Web (nativo)	whatsapp- web.js + Chromium	crm-nativo	Sesión QR persistida	Permanente (proceso dedicado)	Sí
10	SMTP	SMTP	notificacione s (smtp.strateg y.ts)	Config en BD	Por notificación	Sí
11	GenieACS	HTTP NBI	olt-nativo	GENIEACS_NBI_URL + creds de provision	Continua (crons TR-069)	Sí
12	SmartOLT / AdminOLT	HTTPS REST	smartolt , olt-nativo	SMARTOLT_URL , SMARTOLT_TOKEN , olt_proveedor_config	Bajo demanda + sync	Sí (circuit breaker)
13	XUI.ONE (IPTV)	HTTP REST	xui	xui_servidores (cifrado)	30 s	Sí
14	Servidor de licencias	HTTPS + webhook	licencia	LICENSE_KEY + machine-id	Diaria + cada 6 h + webhook	NO — bloquea todo el ERP
15	Let's Encrypt / Certbot	ACME	config + contenedor certbot	Webroot	Cada 12 h	—

#	Integración	Método	Módulo	Credencial	Frecuencia	Degradable
16	Telegram (telegraf)	Bot API	—	—	Dependencia declarada, sin uso detectado	—
17	Twilio	REST	—	—	Dependencia declarada, sin proveedor registrado en el gateway	—

12.2 Integraciones que el prompt asumía y NO existen

Asumido	Estado real
SUNAT / facturación electrónica	No implementada. Existe la página configuracion/facturacion-electronica en el frontend y el módulo facturacion genera PDF con pdfkit , pero no hay cliente de SUNAT, ni OSE, ni firma XML, ni envío de CDR en el backend.
SMS	No existe proveedor de SMS en gateway-mensajeria.service.ts . Las estrategias registradas son WhatsApp nativo, mensajería masiva y SMTP.
Niubiz / Culqi / Webpay / Stripe	No implementadas. Documentadas como pendientes de la Etapa II de cobranza.
Telegram	Librería instalada, sin integración.

12.3 Gestión de credenciales

Ubicación	Contenido	Protección
backend/.env.production	Secretos de infraestructura: DB_PASSWORD , JWT_SECRET , PORTAL_JWT_SECRET , ENCRYPTION_KEY , REDIS_PASSWORD , LICENSE_KEY , tokens de API	Filesystem del VPS, fuera del repo
Base de datos	Credenciales de routers, OLTs, proveedores, servidores XUI, tokens Google	Cifradas con encryption.util.ts (ENCRYPTION_KEY)
ACCESOS.local.mmd	Todas las credenciales del entorno	Solo local — nunca a GitHub ni al VPS
Frontend	Ningún secreto. El proceso PM2 declara entorno mínimo a propósito.	—

Incidente de configuración registrado: hasta 2026-07-22 el proceso del frontend arrastraba todos los secretos del backend (DB_PASSWORD , ENCRYPTION_KEY , JWT_SECRET , REDIS_PASSWORD ...) por haberse lanzado desde una shell con el .env del backend cargado. El frontend es el proceso expuesto y no necesita ninguno: en runtime solo usa NODE_ENV , y sus NEXT_PUBLIC_* se hornean en build.

Dependencia de credenciales duplicadas: las credenciales de connection-request de GenieACS están **hardcodeadas en la configuración de GenieACS** y deben coincidir con el .env de cada VPS (provision erp-connreq-creds). Es un acoplamiento fuera del repositorio.